



Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria en
Ingeniería y Tecnologías Avanzadas.



Reporte de Practica

Base de Datos Distribuidas

Prof.: De La Cruz Sosa Carlos

3TM2

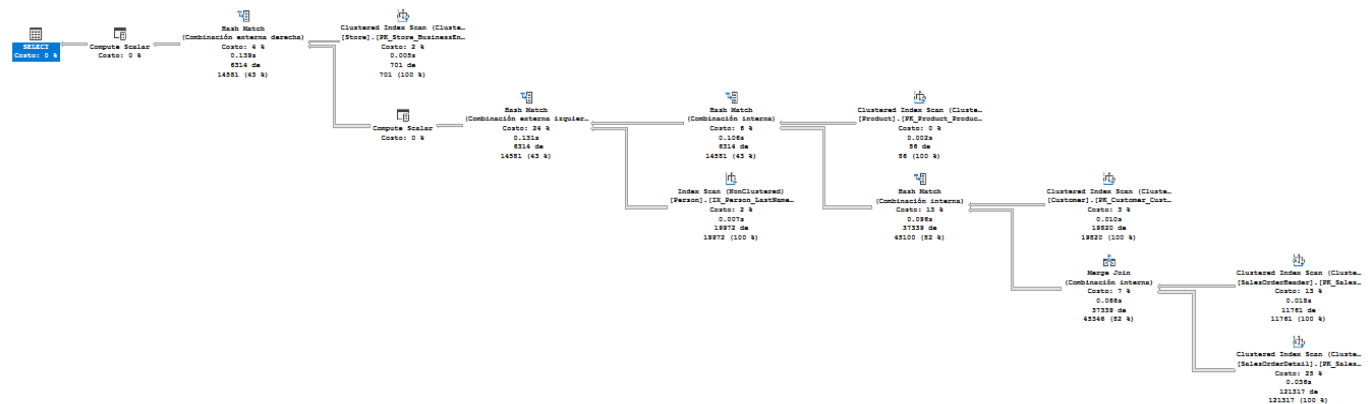
- **Nieves Hernández Efrain**
- **Rodríguez Escalera Ricardo**
- **Sánchez Lemus Ulises Mariano**

Consulta 1

```

SELECT
    p.Name AS Producto,
    sod.OrderQty,
    soh.OrderDate,
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name) AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod
    ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh
    ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c
    ON soh.CustomerID = c.CustomerID
LEFT JOIN Person.Person pe
    ON c.PersonID = pe.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
WHERE YEAR(soh.OrderDate) = 2014
AND p.ListPrice > 1000;

```



Clustered Index Scan (Cluste...
[SalesOrderDetail].[PK_Sales...]
Costo: 25 %
0.036s
121317 de
121317 (100 %)

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page serve
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page serv
Table 'Person'. Scan count 1, logical reads 105, physical reads 1, page serve
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 3,
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3,
Table 'Customer'. Scan count 1, logical reads 123, physical reads 1, page ser
Table 'Product'. Scan count 1, logical reads 15, physical reads 1, page serve
Table 'Store'. Scan count 1, logical reads 103, physical reads 1, page serve

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1	22/03/2026	307 ms	125ms	2270	Clustered Index Scan – 25%	Uso de función sobre tabla	Usar un rango para que no escanee toda la tabla y usar un índice no agrupado que incluya OrderDate y CustomerID.

Consulta optimizada

```

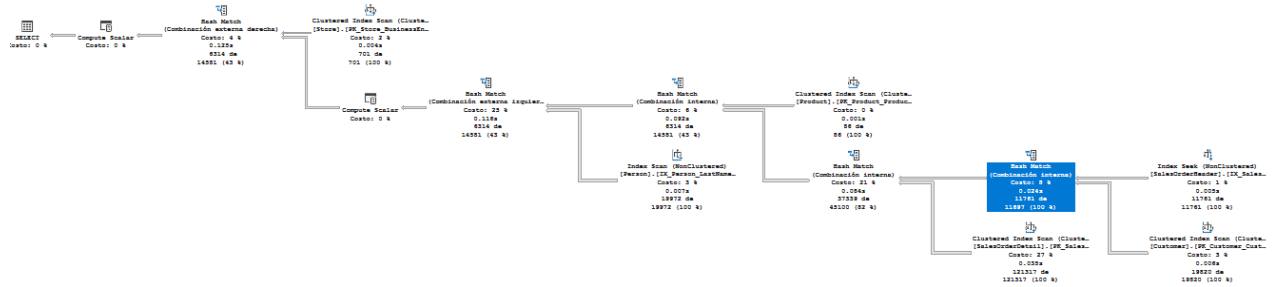
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_OrderDate
ON Sales.SalesOrderHeader (OrderDate)
INCLUDE (CustomerID);

set statistics io on
set statistics time on

SELECT
    p.Name AS Producto,
    sod.OrderQty,
    soh.OrderDate,
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name) AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod
    ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh
    ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c
    ON soh.CustomerID = c.CustomerID
LEFT JOIN Person.Person pe
    ON c.PersonID = pe.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
WHERE
    soh.OrderDate >= '20140101'
    AND soh.OrderDate < '20150101'
    AND p.ListPrice > 1000;

```

Bajo el tiempo de ejecución, de CPU y el número de consultas lógicas.



(6314 filas afectadas)

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0

Table 'Person'. Scan count 1, logical reads 105, physical reads 105

Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 1238

Table 'Customer'. Scan count 1, logical reads 123, physical reads 123

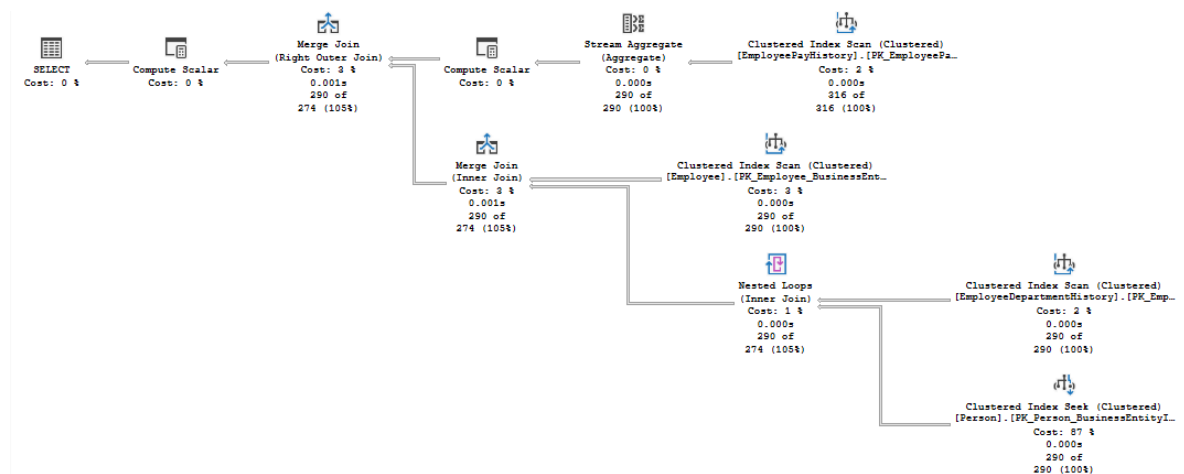
Table 'SalesOrderHeader'. Scan count 1, logical reads 35, physical reads 35

Table 'Product'. Scan count 1, logical reads 15, physical reads 15

Table 'Store'. Scan count 1, logical reads 103, physical reads 103

CONSULTA 2:

```
SELECT e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID,
       (SELECT AVG(rh.Rate) FROM HumanResources.EmployeePayHistory rh
        WHERE rh.BusinessEntityID = e.BusinessEntityID) as PromedioSalario
FROM HumanResources.Employee e
JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID = edh.BusinessEntityID
WHERE edh.EndDate IS NULL;
```





Clustered Index Seek (Clustered)
[Person].[PK_Person_BusinessEntityI...

Cost: 87 %

0.000s

290 of

290 (100%)

(290 rows affected)

Table 'Person'. Scan count 0, logical reads 897, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
Table 'EmployeeDepartmentHistory'. Scan count 1, logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead
Table 'Employee'. Scan count 1, logical reads 9, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
Table 'EmployeePayHistory'. Scan count 1, logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0

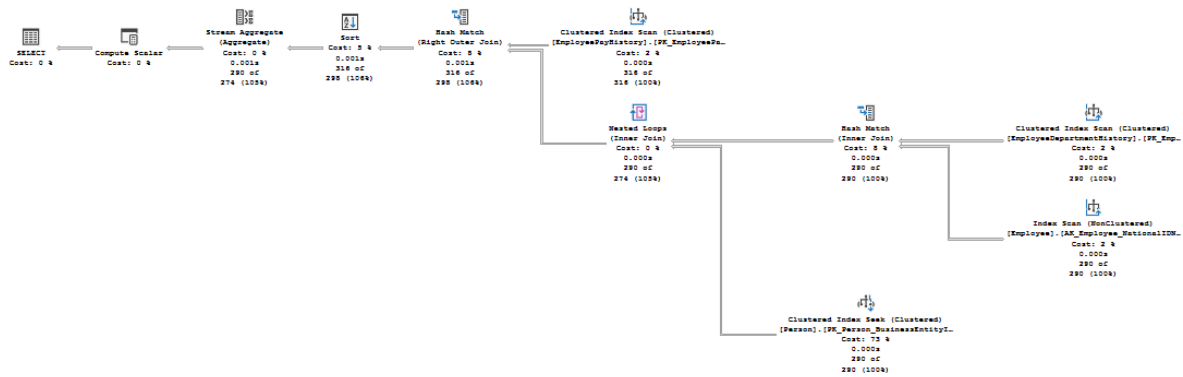
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	23/03/2026	55 ms	0 ms	914	Clustered Index Seek – 87%	Subconsulta correlacionada en el SELECT	Evitar subconsulta, usar en su lugar LEFT JOIN y agrupar con GROUP BY.

Consulta optimizada:

```
set statistics io on  
set statistics time on
```

```
SELECT  
    e.NationalIDNumber,  
    p.FirstName,  
    p.LastName,  
    edh.DepartmentID,  
    AVG(rh.Rate) AS PromedioSalario  
FROM HumanResources.Employee e  
JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID  
JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID = edh.BusinessEntityID  
LEFT JOIN HumanResources.EmployeePayHistory rh ON e.BusinessEntityID = rh.BusinessEntityID  
WHERE edh.EndDate IS NULL  
GROUP BY  
    e.NationalIDNumber,  
    p.FirstName,  
    p.LastName,  
    edh.DepartmentID;
```

Resultado: Disminuyó el tiempo de CPU y se eliminó la ejecución iterativa de la subconsulta, reduciendo las lecturas lógicas globales



(290 rows affected)
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'Person'. Scan count 0, logical reads 897, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'Employee'. Scan count 1, logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'EmployeeDepartmentHistory'. Scan count 1, logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'EmployeePayHistory'. Scan count 1, logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:

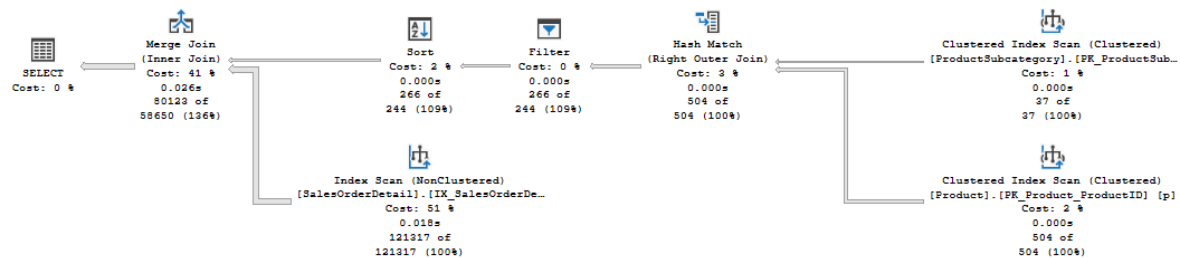
CONSULTA 3:

```
SELECT sod.SalesOrderID, p.ProductID, p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE p.CategoryID = 1 OR p.CategoryID = 2 OR p.CategoryID = 3 OR p.ListPrice > 500;
```

Esta consulta tiene un error de sintaxis ya que hay una columna que no existe por lo que no da plan de ejecución ni tiempo.

Corrección de la Consulta:

```
SELECT sod.SalesOrderID, p.ProductID, p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
-- Aquí agregamos la tabla puente que el profe omitió
LEFT JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID = psc.ProductSubcategoryID
WHERE psc.ProductCategoryID = 1 OR psc.ProductCategoryID = 2 OR psc.ProductCategoryID = 3 OR p.ListPrice > 500;
```



(80123 rows affected)
 Table 'SalesOrderDetail'. Scan count 1, logical reads 273, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'Product'. Scan count 1, logical reads 15, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:
 Table 'ProductSubcategory'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob log:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
3	23/03/2026	521 ms	47 ms	290	Index Scan – 51%	Falta tabla (ProductSubcategory) y uso de múltiples OR	Agregar JOIN a ProductSubcategory, usar IN en lugar de OR, y crear índice compuesto.

Consulta optimizada:

```

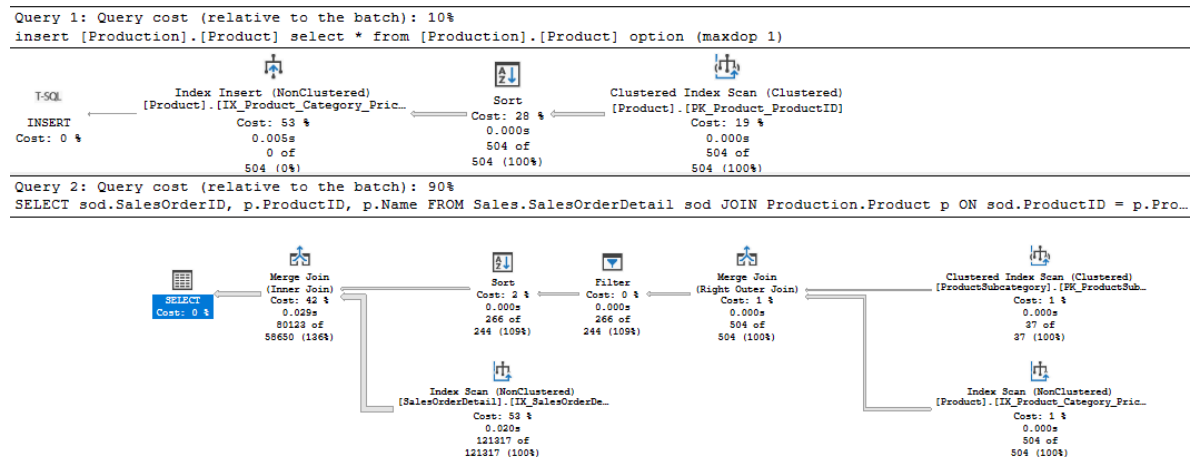
CREATE NONCLUSTERED INDEX IX_Product_Category_Price
ON Production.Product(ProductSubcategoryID, ListPrice)
INCLUDE (Name);

set statistics io on
set statistics time on

SELECT
    sod.SalesOrderID,
    p.ProductID,
    p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
LEFT JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID = psc.ProductSubcategoryID
WHERE psc.ProductCategoryID IN (1, 2, 3)
    OR p.ListPrice > 500;

```

Resultado: Se resolvió el error de relación entre tablas para evaluar la categoría y bajaron las lecturas lógicas en la tabla Product gracias a la evaluación directa con IN y el índice no agrupado.



(80123 rows affected)
Table 'SalesOrderDetail'. Scan count 1, logical reads 273, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0
Table 'Product'. Scan count 1, logical reads 6, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0
Table 'ProductSubcategory'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0

CONSULTA 4:

```
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,
COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate);
```

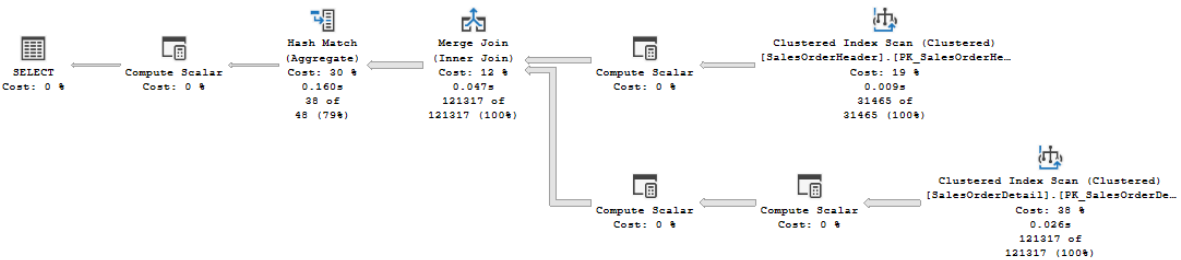


Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 3, page server reads 0, read-ahead reads 1234, page server read-ahead reads 0
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3, page server reads 0, read-ahead reads 682, page server read-ahead reads 0

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
4	23/03/2026	202 ms	156 ms	1924	Clustered Index Scan – 38%	Escaneo completo de tabla de detalles de venta al agrupar y sumar.	Crear índices de cobertura (Covering Indexes) en SalesOrderDetail y SalesOrderHeader.

Consulta optimizada:

```
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_SalesOrderID_LineTotal
ON Sales.SalesOrderDetail(SalesOrderID)
INCLUDE (LineTotal);
```

```
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_OrderDate
ON Sales.SalesOrderHeader(OrderDate);
```

```
set statistics io on
set statistics time on
```

```
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,
       COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate);
```

Resultado: Bajaron drásticamente las lecturas lógicas en SalesOrderDetail al usar un índice de cobertura que incluye LineTotal, y se optimizó el operador con mayor costo.

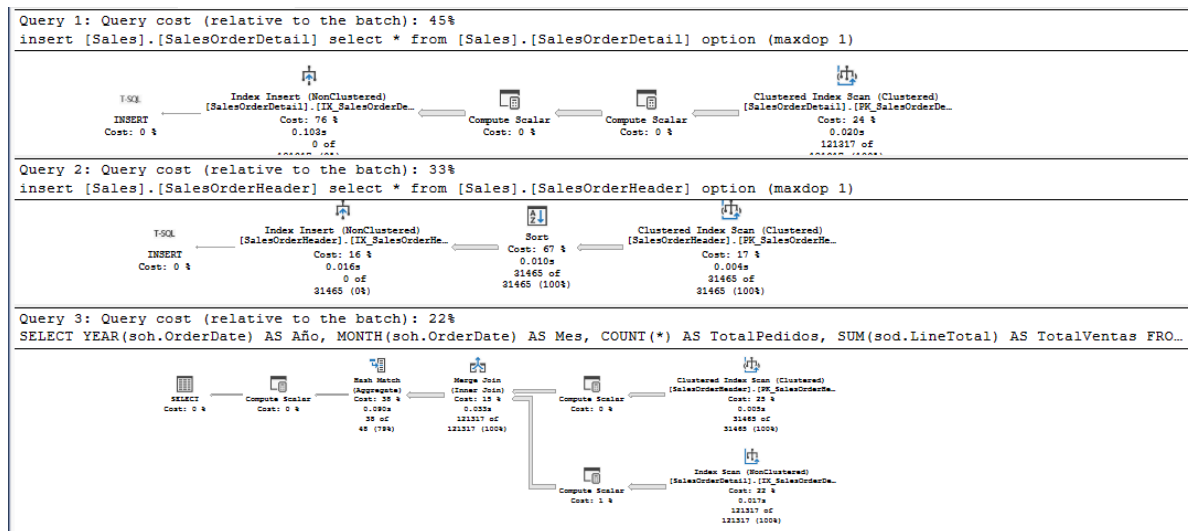


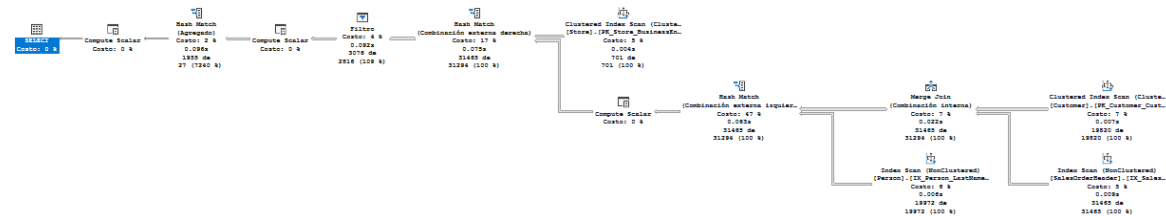
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logi
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logi
Table 'SalesOrderDetail'. Scan count 1, logical reads 467, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0

CONSULTA 6

```

SELECT
    c.CustomerID,
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name) AS Cliente,
    COUNT(*) AS TotalPedidos
FROM Sales.Customer c
JOIN Sales.SalesOrderHeader soh
    ON c.CustomerID = soh.CustomerID
LEFT JOIN Person.Person pe
    ON c.PersonID = pe.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
WHERE UPPER(COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name)) LIKE 'A%'
GROUP BY
    c.CustomerID,
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name);

```



Hash Match
(Combinación externa izquier...
Costo: 47 %
0.063s
31465 de
31294 (100 %)

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page
Table 'Person'. Scan count 1, logical reads 105, physical reads 1, page
Table 'SalesOrderHeader'. Scan count 1, logical reads 57, physical reads
Table 'Customer'. Scan count 1, logical reads 123, physical reads 1, pag
Table 'Store'. Scan count 1, logical reads 103, physical reads 1, page s

(1 fila afectada)

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	23/03 /2026	214 ms	47ms	388	Hash Match – 47%	Uso de UPPER y COALESCE en el WHERE	Eliminar UPPER para no evaluar fila por fila y usar evaluación directa

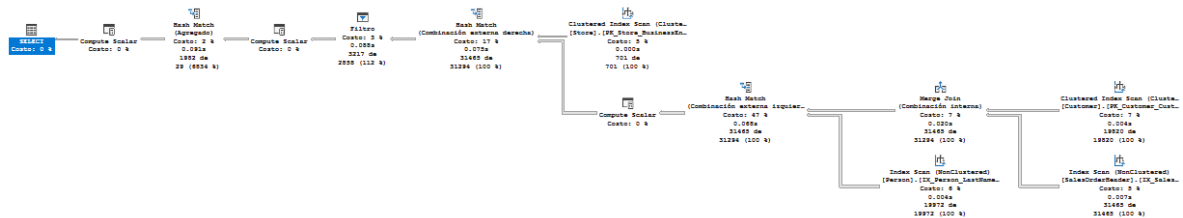
							con LIKE, de esta forma también se evita el uso de COALESCE
--	--	--	--	--	--	--	---

Consulta optimizada

```

SELECT
    c.CustomerID,
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name) AS Cliente,
    COUNT(*) AS TotalPedidos
FROM Sales.Customer c
JOIN Sales.SalesOrderHeader soh
    ON c.CustomerID = soh.CustomerID
LEFT JOIN Person.Person pe
    ON c.PersonID = pe.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
WHERE
    (pe.FirstName + ' ' + pe.LastName LIKE 'A%' OR s.Name LIKE 'A%')
GROUP BY
    c.CustomerID,
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name);

```



Hash Match
 (Combinación externa derecha)
 Costo: 17 %
 0.075s
 31465 de
 31294 (100 %)

CONSULTA 7

```
SELECT TOP 100 sod.SalesOrderDetailID, sod.OrderQty, sod.UnitPrice, soh.OrderDate
FROM Sales.SalesOrderDetail sod
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
ORDER BY soh.ShipDate DESC, sod.OrderQty DESC, sod.UnitPrice DESC;
```

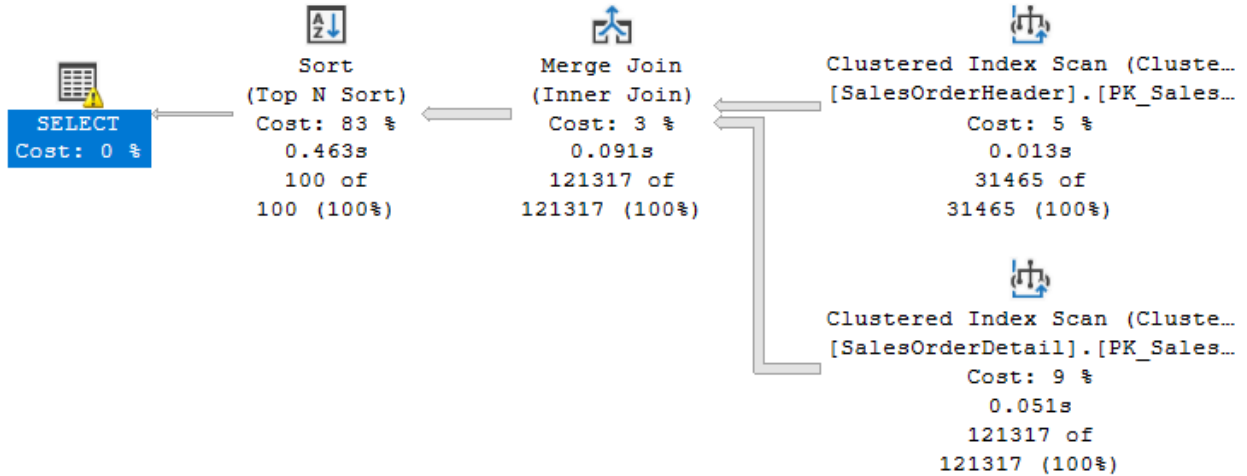


Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1	2026-03-23	471	469	1924	Sort (Top N Sort) - 83%	Ordenamiento costoso, falta de índices.	Índice en ShipDate + columnas ORDER BY.

Consulta optimizada

Crear índices para reducir las lecturas y el uso de CPU.

```
CREATE NONCLUSTERED INDEX IX_ShipDate_Order
ON Sales.SalesOrderHeader (ShipDate DESC)
INCLUDE (SalesOrderID, OrderDate);
```

```
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_Order
ON Sales.SalesOrderDetail (SalesOrderID, OrderQty DESC, UnitPrice DESC)
INCLUDE (SalesOrderDetailID);
```

- El índice en ShipDate es obligatorio para optimizar el ORDER BY.

- El índice en SalesOrderDetail es complementario, mejora el rendimiento del JOIN.
- Ningún índice por sí solo resolverá completamente el ordenamiento porque involucra dos tablas.

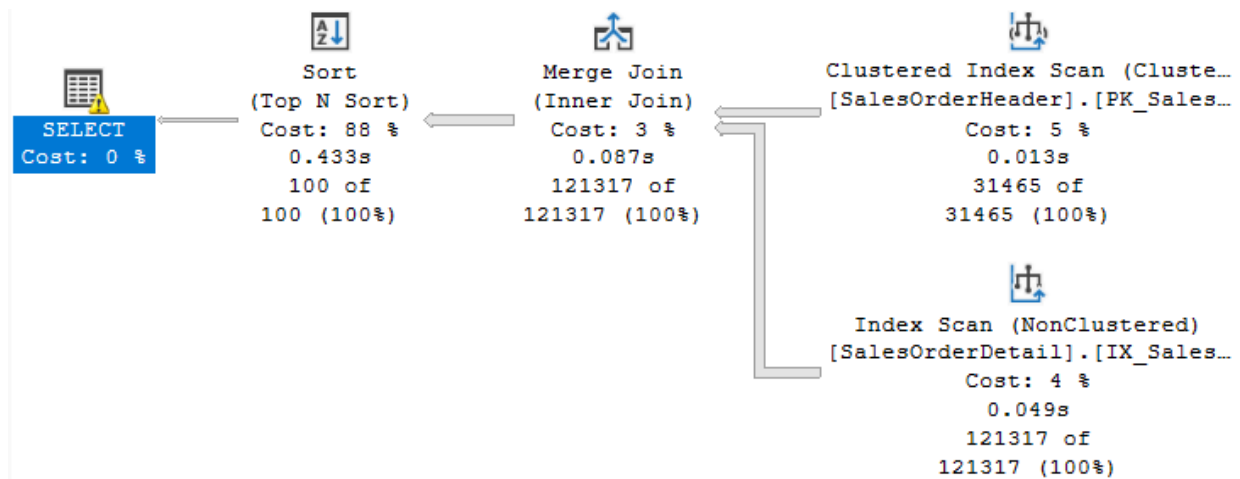


Table 'SalesOrderDetail'. Scan count 1, logical reads 364, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

La consulta presentaba un alto costo debido a operaciones de ordenamiento y escaneo completo. La creación de índices en las columnas utilizadas en el ORDER BY y JOIN permitió reducir lecturas lógicas, CPU y tiempo de ejecución, optimizando significativamente el plan de ejecución.

CONSULTA 8

Se tuvo que hacer una rescritura de la consulta ya que no estaba bien definida y hacía falta un alias sod valido en ese contexto.

```
SELECT p.ProductID, p.Name, SUM(sod.OrderQty) AS TotalVendido
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod
    ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh
    ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.OrderDate >= '2014-01-01'
GROUP BY p.ProductID, p.Name
HAVING SUM(sod.OrderQty) > 100;
```

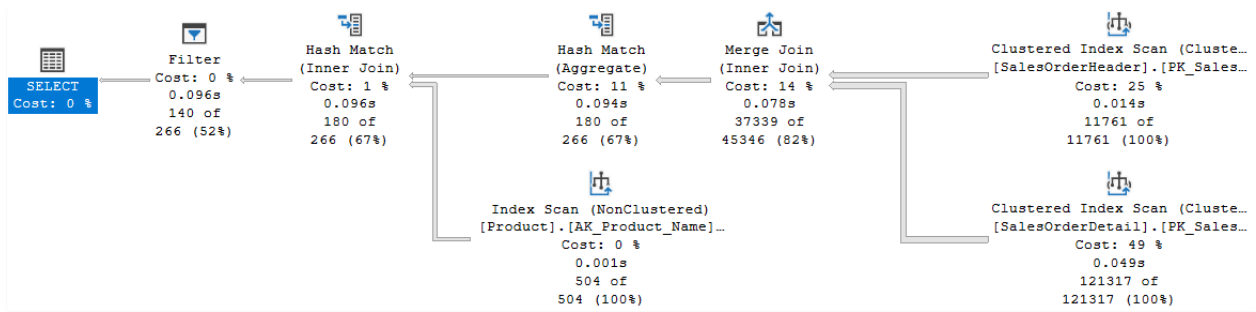


Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Product'. Scan count 1, logical reads 6, physical reads 1, page server reads 0, read-ahead reads 4, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 3, page server reads 0, read-ahead reads 1234, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3, page server reads 0, read-ahead reads 682, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
8	2026-03-23	99	94	1930	Index Scan – 49%	Subconsulta IN, doble agregación.	Reescribir con JOIN + HAVING.

Consulta optimizada

```
CREATE NONCLUSTERED INDEX IX_SOD_Product
ON Sales.SalesOrderDetail (ProductID)
INCLUDE (OrderQty);
```

```
CREATE NONCLUSTERED INDEX IX_SOH_OrderDate
ON Sales.SalesOrderHeader (OrderDate)
INCLUDE (SalesOrderID);
```

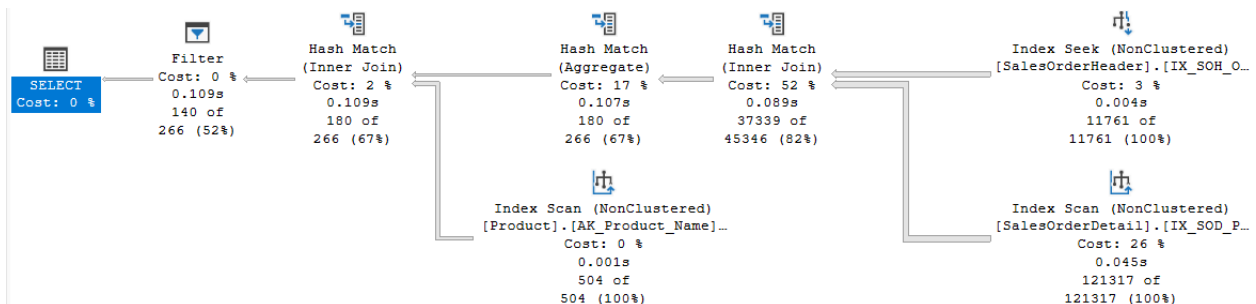


Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'Product'. Scan count 1, logical reads 6, physical reads 1, page server reads 0, read-ahead reads 4, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'SalesOrderDetail'. Scan count 1, logical reads 303, physical reads 1, page server reads 0, read-ahead reads 301, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'SalesOrderHeader'. Scan count 1, logical reads 30, physical reads 1, page server reads 0, read-ahead reads 28, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

La consulta optimizada presenta buen rendimiento, pero la mayoría de las lecturas se concentran en SalesOrderDetail. Se recomienda crear índices sobre ProductID y OrderDate para reducir el costo de acceso y mejorar aún más el rendimiento. Se logro reducir el número de lecturas lógicas menos de 1000.

CONSULTA 9

```
SELECT p.ProductID, p.Name,
       (SELECT COUNT(*) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID = p.ProductID) AS VecesVendido,
       (SELECT SUM(sod.OrderQty * sod.UnitPrice) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID = p.ProductID) AS Ingresos
FROM Production.Product p
JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID = psc.ProductSubcategoryID
WHERE psc.ProductCategoryID = 3;
```

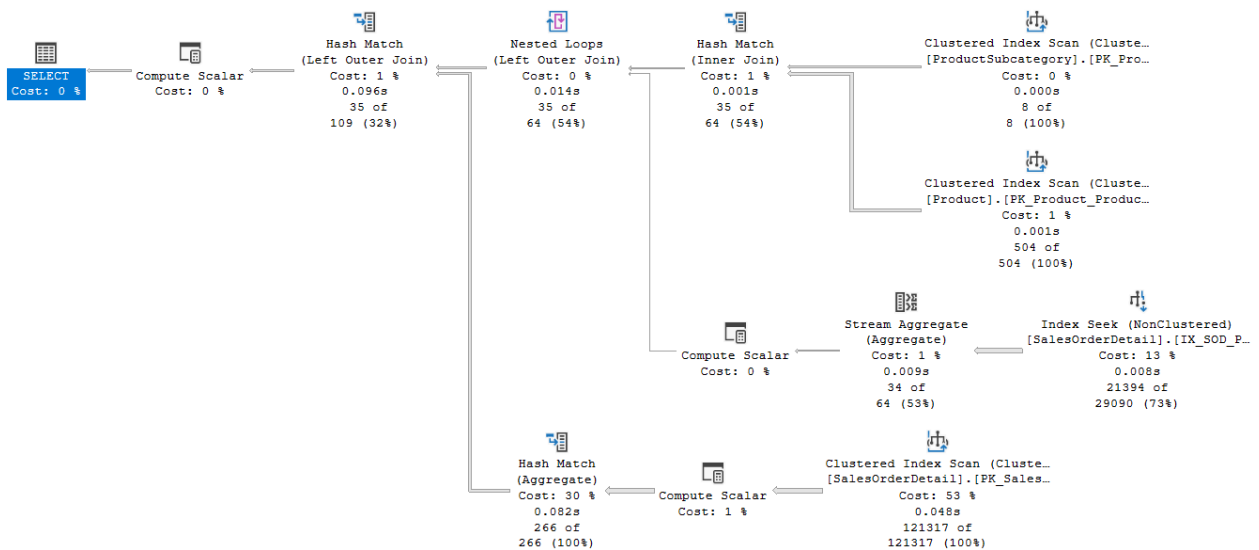


Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'SalesOrderDetail'. Scan count 36, logical reads 1477, physical reads 18, page server reads 0, read-ahead reads 1277, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'Product'. Scan count 1, logical reads 15, physical reads 1, page server reads 0, read-ahead reads 13, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0. Table 'ProductSubcategory'. Scan count 1, logical reads 2, physical reads 1, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Consulta ID	Fecha	Duración (ms)	CP U (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
9	2026-03-23	100	79	1494	Clustured Index Scan – 53%	Subconsultas correlacionadas . Nested Loops.	Reescribir con JOIN + GROUP BY.

						Index Scan repetido.	
--	--	--	--	--	--	----------------------	--

Consulta optimizada

```
CREATE NONCLUSTERED INDEX IX_SOD_Product_Aggregate
ON Sales.SalesOrderDetail (ProductID)
INCLUDE (OrderQty, UnitPrice, SalesOrderDetailID);

SELECT p.ProductID, p.Name,
       COUNT(sod.SalesOrderDetailID) AS VecesVendido,
       SUM(sod.OrderQty * sod.UnitPrice) AS Ingresos
FROM Production.Product p
JOIN Production.ProductSubcategory psc
  ON p.ProductSubcategoryID = psc.ProductSubcategoryID
LEFT JOIN Sales.SalesOrderDetail sod
  ON p.ProductID = sod.ProductID
WHERE psc.ProductCategoryID = 3
GROUP BY p.ProductID, p.Name;
```

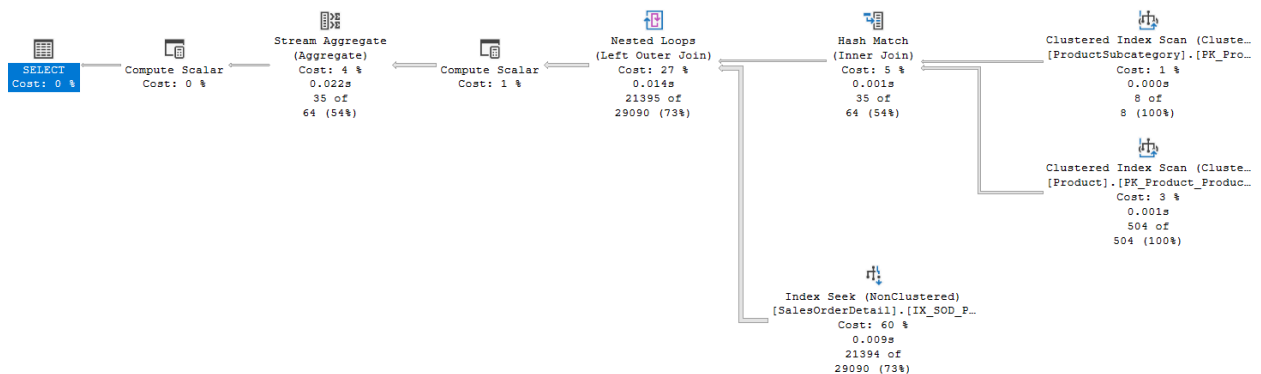


Table 'SalesOrderDetail'. Scan count 35, logical reads 248, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
 Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
 Table 'Product'. Scan count 1, logical reads 15, physical reads 1, page server reads 0, read-ahead reads 13, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
 Table 'ProductSubcategory'. Scan count 1, logical reads 2, physical reads 1, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

La consulta original era ineficiente porque utilizaba subconsultas correlacionadas, lo que provoca que SQL Server ejecute varias veces la misma operación sobre la tabla

SalesOrderDetail, una vez por cada producto. Esto incrementa el número de lecturas y el tiempo de ejecución.

Para mejorar esto, se reescribió la consulta usando JOIN y GROUP BY, lo que permite procesar todos los datos en una sola operación. De esta forma, el motor trabaja de manera más eficiente y reduce el costo de la consulta.

Además, se creó un índice sobre la columna ProductID en la tabla SalesOrderDetail, ya que esta columna se utiliza en la relación entre tablas. Esto permite que SQL Server encuentre los datos más rápido usando Index Seek en lugar de recorrer toda la tabla.

CONSULTA 10

```
SELECT
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name) AS Cliente,
    p.Name AS Producto,
    SUM(sod.OrderQty) AS Cantidad,
    SUM(sod.LineTotal) AS Total,
    DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.Customer c
    ON soh.CustomerID = c.CustomerID
JOIN Sales.SalesOrderDetail sod
    ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
LEFT JOIN Person.Person pe
    ON c.PersonID = pe.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
WHERE
    DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
    AND DATEPART(quarter, soh.OrderDate) = 2
    AND sod.LineTotal > 1000
GROUP BY
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name),
    p.Name,
    soh.OrderDate,
    soh.ShipDate
ORDER BY Total DESC;
```


Clustered Index Scan (Cluste...
[SalesOrderDetail].[PK_Sales...
Costo: 34 %
0.069s
32101 de
32060 (100 %)

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0,
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, ,
Table 'Person'. Scan count 1, logical reads 105, physical reads 1, ,
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical
Table 'SalesOrderHeader'. Scan count 1, logical reads 665, physical
Table 'Customer'. Scan count 1, logical reads 123, physical reads 1
Table 'Product'. Scan count 1, logical reads 6, physical reads 1, p
Table 'Store'. Scan count 1, logical reads 103, physical reads 1, p

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	23/03 /2026	357 ms	172 ms	1640	Clustered Index Scan (SalesOrderDetail) - 34 %	Función DATEDIFF en columna (no SARGABLE), DATEPART sobre columnas, COALESCE dentro de una función (WHERE), GROUP BY mejorable, falta de índice adecuado para la búsqueda.	Evitar la función DATEDIFF sobre columnas, no usar DATEPART, en su lugar usar un rango, mantener COALESCE fuera del WHERE, agregar al GROUP BY los ID's del cliente y el producto, creación de índice en LINETOTAL

Consulta optimizada

```
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_Optimizado
ON Sales.SalesOrderDetail (LineTotal)
INCLUDE (OrderQty, ProductID, SalesOrderID);

SELECT
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name) AS Cliente,
    p.Name AS Producto,
    SUM(sod.OrderQty) AS Cantidad,
    SUM(sod.LineTotal) AS Total,
    DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.Customer c
    ON soh.CustomerID = c.CustomerID
JOIN Sales.SalesOrderDetail sod
    ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
LEFT JOIN Person.Person pe
    ON c.PersonID = pe.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
WHERE
    soh.ShipDate > DATEADD(day, 5, soh.OrderDate)
    AND soh.OrderDate >= '20140101'
    AND soh.OrderDate < '20140701'
    AND sod.LineTotal > 1000
GROUP BY
    c.CustomerID,
    p.ProductID,
    COALESCE(pe.FirstName + ' ' + pe.LastName, s.Name),
    p.Name,
    soh.OrderDate,
    soh.ShipDate
ORDER BY Total DESC;
```



```
Clustered Index Scan (Cluste...
[SalesOrderHeader].[PK_Sales...
Costo: 20 %
0.021s
11761 de
5771 (203 %)
```

```
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0
Table 'Person'. Scan count 1, logical reads 105, physical reads 1
Table 'SalesOrderDetail'. Scan count 1, logical reads 152, physical reads 1
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 1
Table 'Customer'. Scan count 1, logical reads 123, physical reads 1
Table 'Product'. Scan count 1, logical reads 6, physical reads 1
Table 'Store'. Scan count 1, logical reads 103, physical reads 1
```

Bajaron las lecturas lógicas a 1175 y se optimizo el operador con mayor costo.